

FINAL PROJECT STRUKDAT

Kelompok 1

No	Nama	NRP
1	Revalinda Bunga Nayla Laksono	5027251011
2	Putri Permata Sabil	5027251047
3	Nathania Tiara Wahyudi	5027251089
4	Jude Athala Yazid Sari	5027251098
5	Muhammmad Ridwan	5027251113

Opsi 5 - Public Transport Planner

Daftar Isi

1. [Deskripsi Proyek](#)
 2. [Struktur Direktori](#)
 3. [Struktur Data yang Digunakan](#)
 4. [Algoritma yang Diimplementasikan](#)
 5. [Dataset](#)
 6. [Fitur Aplikasi](#)
 7. [Cara Menjalankan Program](#)
 8. [Tracing Manual Algoritma](#)
 9. [Skenario Pengujian](#)
 10. [Analisis Kompleksitas](#)
 11. [Batasan dan Catatan](#)
-

1. Deskripsi Proyek

Proyek ini merupakan simulasi sistem informasi transportasi umum kota Surabaya yang dibangun menggunakan struktur data dan algoritma dasar. Program memodelkan jaringan halte bus dan stasiun kereta sebagai graf berarah berbobot, kemudian menyediakan beberapa fitur pencarian rute yang dapat digunakan oleh pengguna.

2. Struktur Direktori

```
project/
├── src/
│   ├── App.java          -- Entry point, main menu
│   └── model/
```

```

├── Node.java          -- Representasi halte/stasiun
├── Edge.java          -- Representasi rute/koneksi antar node
├── Graph.java         -- Graf adjacency list
├── Trie.java          -- Struktur data Trie
├── MinHeap.java       -- Priority queue untuk Dijkstra
├── TestTrie.java      -- Unit test Trie
├── TestHeap.java      -- Unit test MinHeap
├── algorithm/
│   ├── BFS.java       -- Algoritma BFS minimum transit
│   └── Djikstra.java  -- Algoritma Dijkstra
├── feature/
│   ├── PrefixSearcher.java -- Fitur pencarian halte
│   ├── MinTransitFinder.java -- Fitur BFS dengan tampilan lengkap
│   └── RouteSimulator.java -- Fitur simulasi rute nonaktif
└── data/
    ├── nodes.csv      -- Data halte dan stasiun
    └── edges.csv      -- Data rute antar halte/stasiun

```

3. Struktur Data yang Digunakan

Trie

Prefix tree (Trie) digunakan untuk fitur pencarian halte berdasarkan awalan nama. Setiap node dalam Trie menyimpan satu karakter, dan setiap *path* dari *root* ke node yang ditandai dengan `isEndOfWord = true` merepresentasikan satu nama halte secara utuh. Node akhir tersebut menyimpan referensi ke objek `Node` (model halte) yang bersangkutan.

Visualisasi Trie setelah penambahan tiga nama: "stasiun surabaya gubeng" (S01), "stasiun surabaya pasar turi" (S02), dan "halte diponegoro" (B08). Seluruh nama dikonversi menjadi huruf kecil (*lowercase*) sebelum dimasukkan.

```

root
├── s - t - a - s - i - u - n - (spasi) - s - u - r - a - b - a - y - a - (spasi)
│   └── g
├── u - b - e - n - g [END -> S01]
├── a - s - a - r - (spasi) - t - u - r - i [END -> S02]
├── h - a - l - t - e - (spasi) - d - i - p - o - n - e - g - o - r - o [END -> B08]

```

Ketika user mengetik prefix "**sta**", traversal dimulai dari root melalui `s -> t -> a`, kemudian semua node di bawah subtree tersebut dikumpulkan secara rekursif dan dikembalikan sebagai hasil pencarian.

Ketika prefix tidak ditemukan, misalnya "**xyz**", traversal berhenti di `root -> x -> null` dan fungsi langsung mengembalikan list kosong.

Implementasi: `model/Trie.java`

MinHeap

MinHeap digunakan oleh algoritma Dijkstra sebagai priority queue untuk selalu memproses node dengan bobot terkecil terlebih dahulu. Heap diimplementasikan menggunakan array dengan properti:

- Parent dari node di index i ada di index $(i-1)/2$
- Left child ada di index $2*i + 1$
- Right child ada di index $2*i + 2$

Visualisasi MinHeap setelah insert tiga elemen dengan nilai 20, 5, 12:

```
Insert 20:
Array: [20]

Insert 5:
Array sementara: [20, 5]
Up-heapify: parent(index 1) = index 0, nilai 5 < 20, swap
Array: [5, 20]

Insert 12:
Array sementara: [5, 20, 12]
Up-heapify: parent(index 2) = index 0, nilai 12 > 5, tidak swap
Array: [5, 20, 12]

Representasi pohon:
      5
     / \
    20  12
```

ExtractMin mengambil elemen di index 0 (nilai 5), memindahkan elemen terakhir ke posisi root, lalu menjalankan down-heapify:

```
Setelah extract 5:
Pindah elemen terakhir (12) ke root
Array sementara: [12, 20]
Down-heapify: left child (index 1) = 20, nilai 12 < 20, tidak swap
Array: [12, 20]

Representasi pohon:
      12
     /
    20

Urutan extract: 5, 12, 20
```

Implementasi: `model/MinHeap.java`

Graph (Adjacency List)

Graf direpresentasikan menggunakan adjacency list dengan `HashMap<String, List<Edge>>`. Setiap key adalah ID node, dan value-nya adalah list edge yang keluar dari node tersebut.

Contoh representasi sebagian graf dari dataset:

```
S01 (Stasiun Surabaya Gubeng)
├─ E01 -> S03 | 8 menit | Rp 3.000 | Kereta
├─ E11 -> S02 | 15 menit | Rp 5.000 | Kereta
├─ E52 -> B20 | 10 menit | Rp 4.000 | Bus
└─ E64 -> B06 | 12 menit | Rp 5.000 | Bus

S03 (Stasiun Wonokromo)
├─ E02 -> S01 | 8 menit | Rp 3.000 | Kereta
├─ E03 -> S04 | 12 menit | Rp 4.000 | Kereta
├─ E15 -> S05 | 10 menit | Rp 3.000 | Kereta
└─ E22 -> B01 | 10 menit | Rp 5.000 | Bus

B06 (Halte Tunjungan)
├─ E34 -> B07 | 8 menit | Rp 3.000 | Bus
└─ E36 -> B05 | 5 menit | Rp 3.000 | Bus
```

Graf bersifat directed (berarah) sehingga edge dari A ke B tidak otomatis berlaku sebaliknya. Dataset ini menyediakan edge bolak-balik secara eksplisit, misalnya E01 (S01->S03) dan E02 (S03->S01) adalah pasangan arah yang berlawanan.

Implementasi: `model/Graph.java`

4. Algoritma yang Diimplementasikan

BFS (Breadth-First Search)

BFS digunakan untuk mencari rute dengan jumlah perpindahan halte (transit) paling sedikit. BFS menjamin jalur terpendek dalam satuan langkah karena menjelajah level demi level dari node asal.

Cara kerja:

1. Masukkan path awal [`asalId`] ke dalam queue.
2. Ambil path terdepan dari queue.
3. Untuk setiap tetangga dari node terakhir di path tersebut:
 - Jika tetangga adalah tujuan: kembalikan path ini sebagai hasil.
 - Jika tetangga belum dikunjungi: buat path baru dengan tetangga ditambahkan, masukkan ke queue.
4. Ulangi sampai queue kosong (tidak ada rute) atau tujuan ditemukan.

Catatan implementasi:

- Edge dengan `aktif = false` di-skip, digunakan oleh fitur RouteSimulator.
- Node dengan status `Nonaktif` di dataset juga di-skip.
- Setelah path ditemukan, total waktu dan biaya dihitung terpisah melalui `hitungTotalWaktuBiaya()`.

Implementasi: `algorithm/BFS.java`

Dijkstra

Dijkstra digunakan untuk mencari rute tercepat (berdasarkan waktu) atau termurah (berdasarkan biaya). Algoritma ini menggunakan `PriorityQueue` bawaan Java dan memilih node dengan bobot akumulasi terkecil di setiap langkah.

Cara kerja:

1. Inisialisasi semua jarak ke `Integer.MAX_VALUE`, jarak asal ke 0.
2. Masukkan node asal ke priority queue.
3. Ambil node dengan jarak terkecil dari queue.
4. Untuk setiap edge keluar dari node tersebut, hitung jarak baru. Jika lebih kecil dari jarak yang tersimpan, perbarui dan masukkan ke queue.
5. Ulangi sampai node tujuan diproses atau queue kosong.

Mode yang tersedia:

- "waktu" menggunakan `getWaktuMenit()` sebagai bobot.
- "biaya" menggunakan `getBiayaRupiah()` sebagai bobot.

Implementasi: `algorithm/Dijkstra.java`

5. Dataset

Dataset terdiri dari dua file CSV yang berada di direktori `data/`.

`nodes.csv`

Berisi 30 node (halte dan stasiun). Format kolom: `id, nama, jenis, area, fasilitas, status`

Jenis	Jumlah	Prefix ID
Kereta	10	S01 s.d. S10
Bus	20	B01 s.d. B20

Satu node berstatus `Nonaktif`: B16 (Halte Sidosermo). Node ini tidak akan memiliki edge masuk karena `Graph.addEdge` memvalidasi status node sebelum menambahkan edge.

`edges.csv`

Berisi 65 edge berarah. Format kolom: `id_edge, dari, ke, waktu_menit, biaya_rupiah, jenis_transportasi`

Rentang bobot:

- Waktu: 5 menit s.d. 25 menit
- Biaya: Rp 2.000 s.d. Rp 8.000

Hampir semua edge berpasangan bolak-balik (dua edge terpisah untuk dua arah). Edge yang melibatkan node nonaktif tidak berhasil ditambahkan ke graf saat loading.

6. Fitur Aplikasi

Program dijalankan lewat menu interaktif di terminal.

Menu 1 - Cari halte/stasiun berdasarkan nama Pengguna memasukkan awalan nama halte. Trie mengembalikan semua halte yang namanya dimulai dengan prefix tersebut, lengkap dengan jenis, area, dan status.

Menu 2 - Cari rute tercepat (Dijkstra) Pengguna memasukkan ID asal dan tujuan. Program mengembalikan rute dengan total waktu paling singkat.

Menu 3 - Cari rute termurah (Dijkstra) Sama seperti menu 2, namun optimasi berdasarkan total biaya perjalanan.

Menu 4 - Cari rute minimum transit (BFS) Pengguna memasukkan ID asal dan tujuan. Program mengembalikan rute dengan jumlah perpindahan halte paling sedikit, disertai total waktu dan biaya.

Menu 5 - Bandingkan dua kriteria rute (Dijkstra) Menampilkan rute tercepat dan rute termurah secara berdampingan untuk dibandingkan.

Menu 6 - Simulasi rute tidak tersedia Pengguna dapat menonaktifkan satu atau lebih edge berdasarkan ID edge, lalu mencari rute minimum transit untuk melihat dampaknya. Edge tidak dihapus dari graf, hanya di-flag `aktif = false`. Saat keluar dari menu ini, semua edge dikembalikan ke kondisi aktif secara otomatis.

Menu 7 - Cari halte dan rute sekaligus (Integrasi Trie + BFS) Pengguna mencari halte asal dan tujuan menggunakan prefix nama (Trie), kemudian program langsung menjalankan BFS untuk mendapatkan rute minimum transit.

7. Cara Menjalankan Program

Pastikan Java Development Kit (JDK) versi 11 atau lebih baru sudah terinstal.

```
# 1. Masuk ke direktori src
cd src

# 2. Kompilasi semua file Java
javac -encoding UTF-8 model/*.java algorithm/*.java feature/*.java App.java

# 3. Jalankan program
java -Dfile.encoding=UTF-8 App
```

Untuk menjalankan unit test secara terpisah:

```
# Test Trie
cd src
javac -encoding UTF-8 model/Node.java model/Trie.java model/TestTrie.java
java model.TestTrie

# Test MinHeap
javac -encoding UTF-8 model/Node.java model/MinHeap.java model/TestHeap.java
java model.TestHeap

# Test Dijkstra dengan data kecil
javac -encoding UTF-8 model/*.java algorithm/Dijkstra.java model/TesProgram.java
java model.TesProgram
```

Catatan: Program membaca file CSV dari path relatif `../../data/nodes.csv` dan `../../data/edges.csv` relatif terhadap direktori `src/`. Pastikan struktur direktori sesuai sebelum menjalankan program.

8. Tracing Manual Algoritma

Tracing Trie: Insert dan Search

Data yang diinsert: "stasiun surabaya gubeng" (S01), "stasiun surabaya pasar turi" (S02), "halte diponegoro" (B08)

Semua nama diubah ke lowercase sebelum diproses.

Proses insert "stasiun surabaya gubeng":

Setiap karakter dibuat node baru jika belum ada di `children`. Setelah traversal selesai di karakter terakhir `g`, node tersebut di-set `isEndOfWord = true` dan referensi ke objek Node S01 ditambahkan ke `associatedNodes`.

Search dengan prefix "sta":

Traversal: `root -> s -> t -> a`

Node `a` berhasil ditemukan. Dari sini, `findAllNodes` dijalankan secara rekursif ke seluruh subtree di bawahnya. Semua node yang memiliki `isEndOfWord = true` dikumpulkan dan dikembalikan. Hasil: S01 dan S02.

Search dengan prefix "xyz":

Traversal: `root -> x` menghasilkan `null` karena `x` tidak ada di children root. Fungsi langsung mengembalikan list kosong tanpa traversal lebih lanjut.

Tracing MinHeap: Insert dan ExtractMin

Skenario: insert node Gubeng (nilai 20), Wonokromo (nilai 5), Gedangan (nilai 12)

Step 1: Insert Gubeng (20)

- Array: [Gubeng:20]
- current = 0, tidak ada parent, selesai.

Step 2: Insert Wonokromo (5)

- Array sementara: [Gubeng:20, Wonokromo:5]
- current = 1, parent(1) = 0
- heap[1].value (5) < heap[0].value (20) -> swap
- Array: [Wonokromo:5, Gubeng:20]

Step 3: Insert Gedangan (12)

- Array sementara: [Wonokromo:5, Gubeng:20, Gedangan:12]
- current = 2, parent(2) = 0
- heap[2].value (12) > heap[0].value (5) -> tidak swap
- Array: [Wonokromo:5, Gubeng:20, Gedangan:12]

ExtractMin #1:

- min = Wonokromo:5
- Pindah elemen terakhir ke root: [Gedangan:12, Gubeng:20]
- minHeapify(0):
 - left(0) = index 1 -> Gubeng:20
 - right(0) = index 2 -> tidak ada
 - smallest = 0 (nilai 12 < 20), tidak swap
- Hasil: Wonokromo (5)

ExtractMin #2:

- min = Gedangan:12
- Pindah elemen terakhir ke root: [Gubeng:20]
- minHeapify(0): tidak ada child
- Hasil: Gedangan (12)

ExtractMin #3:

- min = Gubeng:20
- Array: []
- Hasil: Gubeng (20)

Urutan output: Wonokromo (5) -> Gedangan (12) -> Gubeng (20)

Tracing BFS: Minimum Transit**Skenario A: Ada edge langsung**

Asal: S01 (Stasiun Surabaya Gubeng), Tujuan: B06 (Halte Tunjungan) Edge keluar dari S01: E01->S03, E11->S02, E52->B20, E64->B06

Inisialisasi:

- queue : `[[S01]]`
- visited: `{S01}`

Iterasi 1:

- Poll: `[S01]`
- Periksa tetangga S01:
 - S03: belum dikunjungi, masuk queue
 - S02: belum dikunjungi, masuk queue
 - B20: belum dikunjungi, masuk queue
 - B06: B06 == tujuan -> DITEMUKAN

Path final : `[S01, B06]`

- jumlahTransit: 1
- totalWaktu : 12 menit (edge E64)
- totalBiaya : Rp 5.000 (edge E64)

BFS selesai di iterasi pertama karena ada edge langsung dari S01 ke B06.

Skenario B: Tidak ada edge langsung

Asal: B13 (Halte ITS Sukolilo), Tujuan: B07 (Halte Urip Sumoharjo) Edge keluar dari B13: E45->B14, E56->B18, E62->S01

Inisialisasi:

- queue : `[[B13]]`
- visited: `{B13}`

Iterasi 1:

- Poll: `[B13]`
- Tetangga B13:
 - B14 -> queue: `[[B13,B14]]`, visited: `{B13,B14}`
 - B18 -> queue: `[[B13,B14],[B13,B18]]`, visited: `{...,B18}`
 - S01 -> queue: `[[B13,B14],[B13,B18],[B13,S01]]`, visited: `{...,S01}`

Iterasi 2:

- Poll: `[B13,B14]`
- Tetangga B14: B13(visited), B19
 - B19 -> queue: `[..., [B13,B14,B19]]`, visited: `{...,B19}`

Iterasi 3:

- Poll: `[B13,B18]`
- Tetangga B18: B17, B13(visited)
 - B17 -> queue: `[..., [B13,B18,B17]]`, visited: `{...,B17}`

Iterasi 4:

- Poll: [B13, S01]
- Tetangga S01: S03, S02, B20, B06, B13(visited)
 - S03 -> queue tambah [B13, S01, S03]
 - S02 -> queue tambah [B13, S01, S02]
 - B20 -> queue tambah [B13, S01, B20]
 - B06 -> queue tambah [B13, S01, B06]

Iterasi 5:

- Poll: [B13, B14, B19]
- Tetangga B19: B14(visited), B20(visited)
- Tidak ada node baru.

Iterasi 6:

- Poll: [B13, B18, B17]
- Tetangga B17: B18(visited), B11
 - B11 -> queue tambah [B13, B18, B17, B11]

... iterasi berlanjut ...

Iterasi N:

- Poll: [B13, S01, B06]
- Tetangga B06: B07, B05
 - B07 == tujuan -> DITEMUKAN

Path final : [B13, S01, B06, B07]

- jumlahTransit: 3
- totalWaktu : $18 + 12 + 8 = 38$ menit
- totalBiaya : $\text{Rp } 6.000 + \text{Rp } 5.000 + \text{Rp } 3.000 = \text{Rp } 14.000$

BFS menjamin ini adalah path dengan transit paling sedikit yang bisa ditemukan dari B13 ke B07.

Tracing Dijkstra: Rute Tercepat

Skenario: S03 (Stasiun Wonokromo) -> B06 (Halte Tunjungan), mode waktu

Edge relevan dari dataset:

- S03 -> S01: 8 menit (E02)
- S03 -> S04: 12 menit (E03)
- S03 -> S05: 10 menit (E15)
- S03 -> B01: 10 menit (E22)
- S01 -> S02: 15 menit (E11)
- S01 -> B20: 10 menit (E52)
- S01 -> B06: 12 menit (E64)

Inisialisasi:

- jarak[semua] = MAX_VALUE
- jarak[S03] = 0
- PQ : [(S03, 0)]
- visited: {}

Step 1: Poll S03 (jarak=0)

- visited: {S03}
- Update tetangga:
 - S01: $0+8 = 8$, sebelum[S01] = S03
 - S04: $0+12 = 12$, sebelum[S04] = S03
 - S05: $0+10 = 10$, sebelum[S05] = S03
 - B01: $0+10 = 10$, sebelum[B01] = S03
- PQ: [(S01,8), (S05,10), (B01,10), (S04,12)]

Step 2: Poll S01 (jarak=8)

- visited: {S03, S01}
- Update tetangga:
 - S03: $8+8=16 > \text{jarak}[S03]=0$, skip
 - S02: $8+15=23$, sebelum[S02] = S01
 - B20: $8+10=18$, sebelum[B20] = S01
 - B06: $8+12=20$, sebelum[B06] = S01
 - B13: $8+18=26$, sebelum[B13] = S01
- PQ: [(S05,10), (B01,10), (S04,12), (B20,18), (B06,20), (S02,23), (B13,26)]

Step 3: Poll S05 (jarak=10)

- Update: S06 = $10+8 = 18$, sebelum[S06] = S05

Step 4: Poll B01 (jarak=10)

- Update: B11 = $10+12 = 22$, sebelum[B11] = B01

Step 5: Poll S04 (jarak=12)

- Update: S08 = $12+7 = 19$, sebelum[S08] = S04

... iterasi berlanjut sampai B06 di-poll ...

Step N: Poll B06 (jarak=20)

- B06 == tujuan -> SELESAI

Rekonstruksi path dari sebelum[]:

- B06 <- sebelum[B06]=S01 <- sebelum[S01]=S03
- Balik urutan: [S03, S01, B06]

Rute : Stasiun Wonokromo -> Stasiun Surabaya Gubeng -> Halte Tunjungan Total waktu : 20 menit

9. Skenario Penqujian

Skenario Normal

Skenario 1: Rute tercepat dengan jalur kereta langsung

Atribut	Detail
Input	Asal: S01, Tujuan: S09
Fitur	Menu 2, Dijkstra mode waktu
Ekspektasi	Rute: S01 -> S03 -> S04 -> S08 -> S09
Total waktu	$8 + 12 + 7 + 5 = 32$ menit

Jalur ini mengikuti jalur kereta langsung dari Gubeng ke Sidoarjo. Tidak ada jalur alternatif yang lebih cepat berdasarkan bobot edge yang tersedia di dataset.

Skenario 2: Rute minimum transit antara terminal dan halte

Atribut	Detail
Input	Asal: B01, Tujuan: B07
Fitur	Menu 4, BFS
Ekspektasi	Rute melalui node perantara paling sedikit

B01 (Terminal Purabaya) memiliki edge ke S03 dan B11. Dari B11 terdapat edge ke B12, dari B12 ke B10, dari B10 ke B03, dari B03 ke B07. BFS akan menemukan path terpendek dalam satuan hop, bukan dalam satuan waktu atau biaya.

Skenario 3: Pencarian halte dengan prefix lalu rute sekaligus

Atribut	Detail
Input	Prefix asal: "terminal", prefix tujuan: "halte tunj"
Fitur	Menu 7, integrasi Trie + BFS
Hasil Trie asal	B01 (Terminal Purabaya), B02 (Terminal Tambak Osowilangun)
Hasil Trie tujuan	B06 (Halte Tunjungan)
Ekspektasi	User memilih B01, lalu BFS mencari rute B01 -> B06

Edge Case

Edge Case 1: Asal sama dengan tujuan

Atribut	Detail
Input	Asal: S01, Tujuan: S01

Atribut	Detail
Fitur	Menu 4, BFS
Ekspektasi	Rute: [S01], jumlahTransit: 0, totalWaktu: 0, totalBiaya: 0
Penanganan	Ditangani secara eksplisit di awal fungsi <code>cariMinimumTransit</code> sebelum queue diinisialisasi

Edge Case 2: Tujuan adalah node nonaktif

Atribut	Detail
Input	Asal: S01, Tujuan: B16
Fitur	Menu 4, BFS
Ekspektasi	BFS tidak menemukan rute karena B16 tidak memiliki edge masuk
Penanganan	<code>Graph.addEdge</code> melewati penambahan edge jika node asal atau tujuan berstatus Nonaktif . Akibatnya B16 tidak punya edge masuk sama sekali, sehingga BFS tidak akan pernah mencapainya.

Edge Case 3: Semua edge keluar dari node asal dinonaktifkan via simulasi

Atribut	Detail
Langkah	Nonaktifkan E01, E11, E52, E64 (semua edge keluar dari S01) lewat Menu 6
Input	Asal: S01, Tujuan: B06
Fitur	Menu 6 sub-menu 4
Ekspektasi	"Tidak ada rute yang tersedia dengan kondisi simulasi saat ini."
Penanganan	BFS melewati edge dengan <code>aktif = false</code> . Karena tidak ada tetangga yang bisa dijangkau dari S01, queue akan kosong tanpa menemukan tujuan.

Edge Case 4: Prefix tidak cocok dengan nama manapun di Trie

Atribut	Detail
Input	Prefix: "xyz"
Fitur	Menu 1 atau Menu 7
Ekspektasi	"Data tidak ditemukan."
Penanganan	<code>searchByPrefix</code> mengembalikan list kosong saat traversal karakter pertama menghasilkan <code>null</code> di children root.

Edge Case 5: ID node tidak terdaftar di dataset

Atribut	Detail
Input	Asal: "ZZZ", Tujuan: S01
Fitur	Menu 2, 3, 4
Ekspektasi	Menu 4: "ERROR: ID 'ZZZ' tidak ditemukan." Menu 2/3: "Rute tidak ditemukan."
Penanganan	<code>MinTransitFinder</code> dan <code>Dijkstra.cari</code> keduanya memanggil <code>graph.containsNode()</code> sebelum menjalankan algoritma.

Edge Case 6: Graf terfragmentasi, tidak ada jalur antar dua node

Atribut	Detail
Kondisi	Terjadi jika semua edge penghubung antara dua komponen dinonaktifkan via simulasi
Input	Asal: S07, Tujuan: B15 (tidak ada jalur langsung di dataset)
Ekspektasi	BFS mengembalikan <code>ditemukan = false</code> , program mencetak pesan tidak ada rute
Penanganan	BFS mengosongkan queue tanpa mencapai tujuan, lalu mengembalikan <code>HasilBFS</code> dengan <code>ditemukan = false</code> .

10. Analisis Kompleksitas

Operasi	Kompleksitas Waktu	Kompleksitas Ruang	Keterangan
<code>Trie.insert</code>	$O(L)$	$O(L)$	L = panjang nama halte yang diinsert
<code>Trie.searchByPrefix</code>	$O(L + R)$	$O(R)$	R = jumlah node hasil yang dikembalikan
<code>MinHeap.insert</code>	$O(\log N)$	$O(1)$	N = jumlah elemen di heap saat itu
<code>MinHeap.extractMin</code>	$O(\log N)$	$O(1)$	Down-heapify maksimal sedalam $\log N$
<code>BFS.cariMinimumTransit</code>	$O(V + E)$	$O(V \times P)$	P = panjang path rata-rata yang disimpan di queue
<code>Dijkstra.cari</code>	$O((V + E) \log V)$	$O(V)$	Menggunakan PriorityQueue Java
<code>Graph.addNode</code>	$O(1)$ amortized	$O(1)$	HashMap put
<code>Graph.getNeighbors</code>	$O(1)$	$O(1)$	HashMap get

Catatan tentang kompleksitas ruang BFS: implementasi ini menyimpan seluruh path di setiap elemen queue, bukan hanya ID node terakhir. Hal ini membuat kompleksitas ruang menjadi $O(V \times P)$ di mana P adalah panjang path rata-rata. Pendekatan alternatif yang lebih efisien adalah menyimpan peta `parent[]` seperti yang dilakukan Dijkstra, namun untuk dataset ini dengan 30 node dan 65 edge, perbedaannya tidak signifikan.

11. Batasan dan Catatan

- **Format Antarmuka:** Program dibangun sepenuhnya dengan *Command Line Interface* (CLI) dan tidak menyediakan *Graphical User Interface* (GUI).
- **Path Data:** Sistem membaca file CSV secara relatif dari direktori `src/` (`../.././data/`). Jika program dijalankan dari lokasi berbeda, alokasi direktori pada `App.java` memerlukan penyesuaian ulang.
- **Implementasi Algoritma:** Struktur `MinHeap` kustom pada `model/MinHeap.java` difungsikan khusus untuk demonstrasi dan *unit testing*. Algoritma utama pada `Dijkstra.java` menggunakan struktur `PriorityQueue` bawaan Java.
- **Simulasi Rute:** Modifikasi *edge* pada fitur simulasi bersifat temporer (mengubah *flag aktif* = `false`) tanpa menghapus objek secara fisik. Kondisi graf akan dinormalkan sepenuhnya saat keluar dari menu.
- **Validasi Masukan:** Sistem validasi *error* pada antarmuka CLI bersifat mendasar. Kesalahan pengetikan (misal: memasukkan karakter huruf saat instruksi meminta angka) akan menyebabkan *exception* `NumberFormatException`.
- **Skala Dataset:** Himpunan graf (30 node dan 65 *edge*) dibangun secara khusus untuk mengakomodir keperluan akademik dan validasi algoritma. Dataset ini tidak merepresentasikan infrastruktur transportasi Kota Surabaya secara nyata dan menyeluruh.